

Foundations for programming languages

12: Functional programming

Isao Sasano

ML

- ML is a functional language with imperative features.
(function-oriented imperative language)
- Features of functional languages: functions are first class values
 - A function can be created using an expression
 - Functions can take functions as their arguments.
 - Functions can return functions.
- Type system in ML is considered to be most beautiful and have much expressive power.
- ML supports important features in Lisp-like or Algol-like languages.
- We illustrate functional languages by using Standard ML. Other functional languages include OCaml, Haskell, etc.

Type system in ML

- Type system in ML is safe.
 - When type checker judges that an expression has some type, it is guaranteed that evaluating the expression results in a value of the type.
 - (ex.) When an expression is a pointer to a string, the value of the expression is guaranteed to be a pointer that points to some memory area that holds a string. (The value is guaranteed not to be a pointer that points to some memory area that have been already freed (dangling pointer) or a pointer that points to some memory area that holds a value other than a string.

Interactive session

- *<exp>;*
val it = <value representation> : <type>

(ex.)

- 5+3-2;

val it = 6 : int

- it+3;

val it = 9 : int

- if true then 1 else 5;

val it = 1 : int

- 5=4;

val it = false : bool

Evaluating (executing) expressions in ML

- Expressions are parsed, type-checked, compiled, and then executed.
- When an expression has some syntax error or type error, code are not generated and the expression is not executed.
- (ex.) The expression `if true then 3 else false` does not have syntax error but have type error since the type system of ML requires for the then-part and else-part have the same type.

- `if true then 3 else false;`

`stdIn:5.1-5.26 Error: types of if branches do not agree [literal]`

`then branch: int`

`else branch: bool`

`in expression: if true then 3 else false`

Declarations

- `val <id> = <exp>;`

`val <id> = <value representation> : <type>`

“val” is a prefix of “value”. The expression `<exp>` is evaluated and the result is bound to the identifier `<id>`.

(ex.)

- `val x = 7+2;`

`val x = 9 : int`

- `val y = x+3;`

`val y = 12 : int`

- `val z = x*y-(x div y);`

`val z = 108 : int`

(Just for reference)

- In ML, / is used for division of values of type real (floating point number).
- Unlike C, implicit conversion is not done in ML.

(ex.)

- 9/12;

stdIn:1.2-1.6 Error: operator and operand
don't agree [literal]

operator domain: real * real

operand: int * int

in expression: 9 / 12

Function declarations

- fun <id> <arg₁> ... <arg_n> = <exp>;
val <id> = fn : <type₁> -> ... -> <type_n> -> <result type>

(ex.)

- fun f (x) = x+5;

val f = fn : int -> int

The function of type int -> int is bound to the identifier f.

- val f = fn x => x+5;

val f = fn : int -> int

This is equivalent to the above declaration.

fn x => x+5 corresponds to the lambda abstraction

$\lambda x. x + 5$.

Assignment

In ML we cannot change the values of variables. For example, under the declaration `val x = 3`, the value of `x` is always 3. (ML declarations introduce variables as constants.) In ML we use reference cells, which enable us to change the values.

```
(ex.)  - val x = ref 3;
        val x = ref 3 : int ref
        - !x;
        val it = 3 : int
        - x:=4;
        val it = () : unit
        - !x;
        val it = 4 : int
```

Identifiers in C, Pascal, etc.

- In C and Pascal, variables of type int are mutable, but in the case of a function declaration, the function name is a constant and cannot be changed to other function through assignment. In other words, in C and Pascal, whether an identifier is a constant or not is determined by its type.

Types

- A type consists of a set of values and operations on them.
- A type is represented by a type expression.

`<type-exp> ::=`

Syntax of (subset of)
type expressions in
ML

`<type-name>`
| `<type-exp> -> <type-exp>`
| `<type-exp> * <type-exp>`
| `<type-exp> list`
| `{<id>:<type-exp>, ..., <id>:<type-exp>}`

Basic types

- When a value cannot be decomposed (in language level), we say it is atomic and its type is basic.
 - (ex.) Values in a set {true, false} are atomic.
- Operations on values of basic types are defined in each type
 - (ex.) Operations on values of int type: $2+3$, $5*6$, $2=2$, $2\neq 2$, etc.

Basic types in ML

- Unit type
 - `() : unit`
 - `()` is used as a return value of functions that do not need to return a value, or as an argument of functions that do not need to take an argument. (Unit type roughly plays a role similarly to void type in C.)
- Boolean type
 - `true: bool, false : bool`
 - In if expression `if  $e_1$  then  $e_2$  else  $e_3$` , e_1 must have bool type, e_2 and e_3 must have the same type. ML does not provide if expression without having else part.
 - Boolean operators `andalso`, `orelse`, `not` correspond to `&&`, `||`, `!` in C.)

Basic types in ML (cont.)

- int type
 - $0, 1, 2, \dots, -1, -2, \dots$: int
 - $+, -, *, \text{div} : \text{int} * \text{int} \rightarrow \text{int}$ (infix operators)
- string type
 - Symbols surrounded by double quotes
 - Concatenation operator $^ : \text{string} * \text{string} \rightarrow \text{string}$ (infix)
- real type
 - $1.0, 2.0, 3.14159, 4.4444, \dots$: real
 - $+, -, *$ are used on either real or int type. (Both sides must have the same type.)

The function “real” converts int type to real type. For example, `real 3` is evaluated to `3.0`. ML does type inference, so explicit conversion is necessary.

Types and product types

- The product type $A * B$ of two types A and B consists of pairs of values of type A and B .
 - (ex.) $(1, \text{"one"})$ is a pair of 1 and "one".
- The product type $A_1 * A_2 * \dots * A_n$ of n types consists of tuples of the form $(a_1, a_2, \dots, a_n)$ where a_i is a value of type A_i with $1 \leq i \leq n$.
- Operations on a product type are a function that takes the first element and a function that takes the second element from a pair.

fun first $(x, y) = x$;

fun second $(x, y) = y$;

Operations on n tuples are defined similarly.

Examples of tuples

Pairs, triples, quadruples, ...

(ex.) - (3, 4);
 val it = (3,4) : int * int
 - (4, 5.5, true);
 val it = (4,5.5,true) : int * real * bool
 - ("Taro", "Jiro", 5);
 val it = ("Taro","Jiro",5) : string * string * int
 - #2 (3, 4);
 val it = 4 : int
 - #1 ("Taro", "Jiro", 5);
 val it = "Taro" : string

Records

Records in ML correspond to structures in C and records in Pascal. Records and tuples are similar but records have names in each

```
(ex.)  - {first_name="Donald", last_name="Knuth"};  
        val it =  
          {first_name="Donald", last_name="Knuth"}  
          : {first_name:string, last_name:string}
```

Lists

- A list is a sequence of elements of same type.
- The type **A list** consists of a set of lists having elements of type **A**.
 - (ex.) **int list** type consists of lists of integers.
- A list is represented by writing elements delimited by commas and surrounded by brackets [and]. The empty list is written by [] or **nil**.
 - (ex.) [1,2,3] is a list of three integers 1,2,3 and ["red", "white", "blue"] is a list of three strings "red", "white", "blue".
 - Cons is written as ::
 - (ex.) - 3::nil;
val it = [3] : int list
- 4::5::it;
val it = [4,5,3] : int list

Lists (cont.)

Unlike Lisp, lists in ML must have elements of the same type.

(ex.) - [1,2,3,4];
 val it = [1,2,3,4] : int list
 - [true,false];
 val it = [true,false] : bool list
 - ["red","blue"];
 val it = ["red","blue"] : string list
 - [fn x=>x+1, fn x=>x+2];
 val it = [fn,fn] : (int -> int) list

Operations on lists

`null(x)` returns true when `x` is the empty list and false otherwise.

`hd(x)` returns the first element in the list.

`tl(x)` returns the list `x` where the first element is deleted.

`a::x` returns the list where `a` is consed to the list `x`.

(ex.) `null [ ]` returns true.

(ex.) `null [1,2,3]` returns false.

(ex.) `[1,2,3] = 1::[2,3] = 1::2::[3] = 1::2::3::[ ]`

`::` is right associative and thus `1::2::[3]` is equivalent to `1::(2::[3])` and `1::2::3::[ ]` is equivalent to `1::(2::(3::[ ]))`.

Patterns

We can use a pattern in the LHS of a val declaration.

```
val <pattern> = <exp>;
```

$\langle \text{pattern} \rangle ::= \langle \text{id} \rangle \mid \langle \text{tuple} \rangle \mid \langle \text{cons} \rangle \mid \langle \text{record} \rangle \mid \langle \text{constr} \rangle$

$\langle \text{tuple} \rangle ::= (\langle \text{pattern} \rangle, \dots, \langle \text{pattern} \rangle)$

$\langle \text{cons} \rangle ::= \langle \text{pattern} \rangle :: \langle \text{pattern} \rangle$

$\langle \text{record} \rangle ::= \{ \langle \text{id} \rangle = \langle \text{pattern} \rangle, \dots, \langle \text{id} \rangle = \langle \text{pattern} \rangle \}$

$\langle \text{constr} \rangle ::= \langle \text{id} \rangle (\langle \text{pattern} \rangle, \dots, \langle \text{pattern} \rangle)$

(Note 1) A same pattern variable cannot appear twice in a pattern. (It cannot be expressed as BNF and thus checked in some other phase than the parsing phase.)

(Note 2) nil, which belongs to $\langle \text{constr} \rangle$, is a constructor with no arguments.

Examples of patterns

- val t = (1,2,3);
val t = (1,2,3) : int * int * int
- val (x,y,z) = t;
val x = 1 : int
val y = 2 : int
val z = 3 : int
- val a::b = [1,2,3];
val a = 1 : int
val b = [2,3] : int list
- val (a,b::c) = (1,[2,3]);
val a = 1 : int
val b = 2 : int
val c = [3] : int list

Function declarations with patterns

```
fun <id> <pattern> = <exp>
```

```
fun <id> <pattern> = <exp>  
  | ...  
  | <id> <pattern> = <exp>
```

A function can be declared in either of these two forms.

(ex.) - fun f (x,y) = x+y;
 val f = fn : int * int -> int
 - fun length nil = 0
 | length (x::xs) = 1 + length xs;
 val length = fn : 'a list -> int

(Note) The function f seems to take two arguments, but actually it takes one argument, which is a pair.

(Cont.)

```
- length ["a","b","c"];  
val it = 3 : int
```

The declaration of function `length` consists of two clauses, and the actual argument is pattern-matched from the first clause. When the argument is `nil`, `length` returns 0. Otherwise the argument is matched with the pattern `x::xs`.

We use datatype declarations to declare a datatype.

(ex.) `datatype tree = LEAF of int | NODE of (tree * tree)`

```
- fun inTree(x,LEAF y)=x=y  
  | inTree(x,NODE(y,z))=inTree(x,y) orelse inTree(x,z);  
val inTree = fn : int * tree -> bool
```